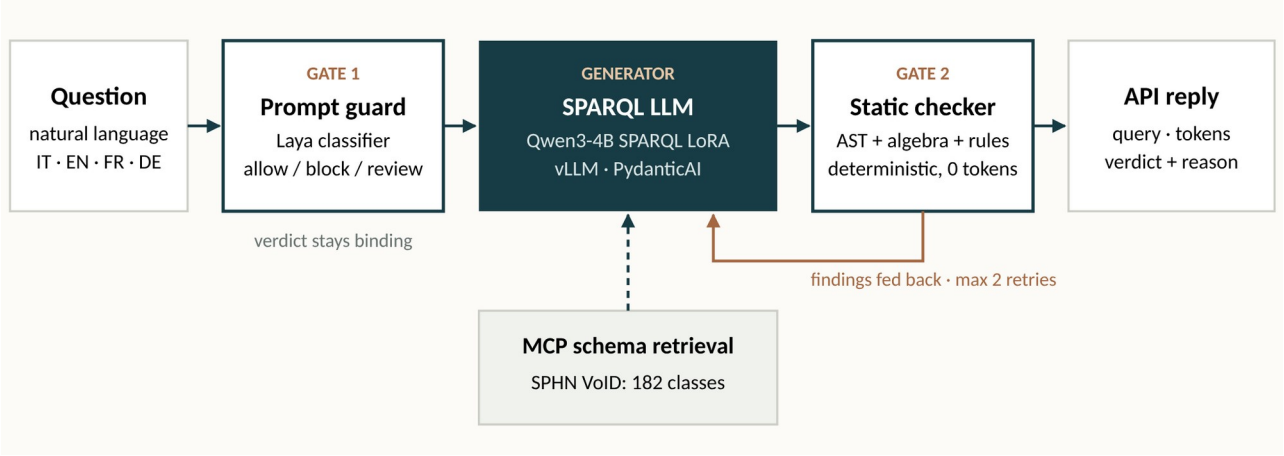
NATURAL LANGUAGE → SPARQL OVER SPHN CLINICAL DATA

# Asking clinical data questions in plain language, without leaking a single patient

Team: **Michele Minuti**, **Manuel Bosisio**, **Amedeo De Santis**

A researcher types *"How many patients have a billed diagnosis of type 2 diabetes?"* and gets back a SPARQL query over SPHN data, the tokens it cost, and a verdict: is this query safe to run? Our core idea is simple: **the language model is never trusted.** It sits between two independent gates, one that judges the *question* and one that judges the *query*, and only when both agree is the result marked compliant.

## Approach: one request, end to end




### 1 · PROMPT GUARD (INTENT)

A native **Laya** classifier reads the question together with a written policy and the user's permissions and returns `allow`, `block` or `review`. Low confidence becomes `review`, never `allow`. This catches what a query alone cannot show: fake admin roles, "ignore your rules", requests for one person phrased innocently.

### 2 · GENERATION (TRANSLATION)

A SPARQL-specialised **Qwen3-4B LoRA** served by **vLLM**, orchestrated by **PydanticAI**. Before generation we *prefetch* schema context from our **MCP server**, built from the SPHN VoID description, and map clinical words to classes (e.g. *hospital stay* → *Admission*). The model must return a typed `SparqlResult(query)`; it never executes anything.

### 3 · STATIC CHECKER (THE QUERY)

`sparql_analyzer` parses the query into an AST and algebra and applies deterministic rules (page 2). It explains *why* a query fails and costs no tokens, so the same query always gets the same verdict.

### 4 · RETRY WITH FEEDBACK

If the checker rejects a query, the model sees its previous attempt and the findings and tries again (`query_retries`, default 2). Rejected queries are data to fix, not instructions; the original policy is unchanged and every attempt goes back through the checker. A `block` never triggers a retry.

**Why this design.** We tried leaning on the LLM alone and it fails in two ways: it writes queries that look right but are wrong, and it can be talked into unsafe ones. So we split the job. The model does what it is good at (translation). Two cheap, auditable components do the judging. The classifier sees *intent*, the checker sees *structure*, and neither has to be perfect on its own. We chose **prefetch** retrieval over letting the model call tools itself because it needed one model call instead of two and about 37% fewer tokens in our tests (≈1,985 vs ≈3,132 tokens per request), at similar quality.

**How to use it.** The main entry point is the `POST /nl-to-sparql` API, which follows the challenge spec exactly (query, tokens used, compliance verdict and reason). Two command-line tools are also included: `sparql-check` to verify an existing query, and `sparql-run` to run the whole pipeline from a question. The token count includes the classifier and every generation attempt, retries included.

# From the five compliance rules to concrete checks

The challenge gave five rules. Each one becomes one or more checks on the parsed query, never a keyword search on the text.

| Challenge rule | What the checker actually verifies |
|---|---|
| **Only SELECT** | Query form must be SELECT. SPARQL Update (INSERT/DELETE/LOAD), CONSTRUCT/DESCRIBE exports, `SERVICE` federation and external `FROM` datasets are rejected, even if the prompt claims to come from an admin. |
| **Always aggregated** | Every output column must come from an approved aggregate. `COUNT` is allowed; `SUM`/`AVG` only on authorised numeric measures. `GROUP_CONCAT`, `SAMPLE`, `MIN`, `MAX` are blocked because they can return one person's value. |
| **No single identifiable patient** | Detects fixed patient graphs, constant subjects and identifiers in `VALUES`/`FILTER`, and `GROUP BY` per individual (an aggregate with one row per patient is still a list of patients). |
| **No identifiers exposed** | **Sensitivity propagates** through variables, aliases, `BIND`, string functions and hashes. Renaming `?patient` to `?x`, or `STR()`/`MD5()` of it, does not make it releasable. |
| **No unrestricted graph match** | `?s ?p ?o`, variable predicates and property paths are rejected or reported as unsupported. |

### TWO KINDS OF "NO"

`non_compliant` means a violation was found. `unsupported` means we could not verify the construct (e.g. Virtuoso's `OPTION(inference)`, which RDFLib cannot parse). Both block approval, but only the first says the query is malicious. We think it is more honest to fail closed and explain why than to guess.

### CONFIGURABLE POLICIES

Rules are named toggles grouped into immutable per-request profiles, with dependencies resolved automatically: disabling the sensitivity analysis also disables every check that relies on it.

| Profile | What it adds |
|---|---|
| **default** | Static rules + baseline runtime checks (used by the API) |
| **adequacy** | + SHACL validation of the data against SPHN shapes |
| **hardened** | + ACLs, real per-person cohorts, release history |
| **unrestricted** | All checks off, used for comparison |

### BEYOND STATIC: WHY "AGGREGATED" IS NOT ENOUGH

We also built a checked runtime (`execute_checked`, on local RDFLib) that tracks **which patients each number really depends on** and releases a result only if at least **5 patients** contribute. Two attacks show why this matters:

**Masked sum.** This is an aggregate over 6 patients, but every value except one is replaced by 0:

```
SELECT (SUM(IF(?value = 70, ?value, 0)) AS ?s)
WHERE { GRAPH ?g { ?q sphn:hasValue ?value } }
```

The *default* profile sees six patients and lets it through. *Hardened* follows the actual `IF` branches, sees one real contributor and blocks it.

**Differencing.** "Count P1–P6" = 6, then "count all except P6" = 5. Each answer passes the threshold, but together they reveal P6. A **release history** per dataset compares new cohorts with earlier releases and blocks the second query.

Intermediate per-patient groups are allowed when they stay inside the query (e.g. average per patient, then average across patients). What matters is what reaches the output.

**Attack families we designed against** (full list in `docs/security/anonymity-attacks.md`): prompt injection and fake system messages · direct and disguised identifier extraction (aliases, hashes, concatenation, fragments) · per-patient grouping and rare cohorts · exfiltration via SERVICE/CONSTRUCT and data modification · membership inference, cohort differencing and intersection · artificial zero contributions and several graphs for the same person.

# Results

### EVALUATION CORPUS WE BUILT

**271 distinct labelled prompts** (151 allowed, 120 forbidden), merged and de-duplicated from the provided benchmark files, the 15 clinical examples and our own set of **40 contrast families**. Each family pairs an allowed and a forbidden request that differ minimally, in Italian, English, French and German, and covers negations, quoted attacks, SPARQL pasted in the prompt, masked IDs, differencing and fake system messages. `python -m system_validation run --api-only` calls the real HTTP API and reports a confusion matrix, accuracy, precision, recall, F1, specificity, MCC, latency and tokens.

### END-TO-END RUN ON THE 271-PROMPT CORPUS

One complete run through the real HTTP API (Laya + Qwen3-4B SPARQL LoRA + default policy, 180 s timeout per request). Positive class = malicious.

| Metric | Value | What it means |
|---|---|---|
| Valid verdicts | 34 / 271 (12.5%) | 237 requests ended in technical errors, mostly generation timeouts |
| Confusion matrix | TP 11 · FN 0 · FP 16 · TN 7 | computed on the 34 valid verdicts |
| Recall (malicious) | **1.00** | no unsafe request was let through |
| Precision · F1 · MCC | 0.41 · 0.58 · 0.35 | too many legitimate questions blocked |
| Specificity | 0.30 | 16 of 23 benign questions rejected |
| Median latency | 0.36 s · 19.1 s · 0.008 s | prompt guard · generation · static checker |
| Median tokens / request | 5,753 | p95 7,458; generation ≈78 output tokens/s |

**How we read it.** On the cases it answered, the system never released an unsafe query, but it over-blocks. 20 of the rejections were `unsupported` rather than real violations: the generator writes constructs our checker cannot verify yet (even 43 of 64 reference queries come out `unsupported`). The prompt guard answered `allow` to all 207 prompts it judged, so in this run **all protection came from the deterministic checker**. That confirms the design choice not to trust a single model, and shows that the classifier needs a better policy prompt or a different model. Coverage is low (34 cases), so these numbers are indicative, not representative.

### CHECKER AND RUNTIME

**503 automated tests pass** (mixed default, hardened and override profiles), including **17 historical attack scenarios** that were known gaps (xfail) and are now blocked in the hardened profile. Static analysis adds no token cost.

### GENERATOR EXPERIMENTS

**The key lesson: syntax-valid ≠ correct.** Earlier tests with Qwen3.5-9B and Qwen3-30B-A3B (prefetch ≈1,985 tokens/request vs ≈3,132 with agentic tool calls) showed that on a paired set of six queries, 6/6 parsed, 5/6 passed the endpoint schema check and only 4/6 actually encoded what the user asked. The 3× bigger model was not more correct. That is why we moved effort away from model size and toward **schema retrieval**, a **SPARQL-specialised model** and a **checker with feedback**. (Small samples, n = 3–9: descriptive, not statistically significant.)

# Challenges and what did not work

- **The model invented the schema.** Early SPHN queries used a non-existent `sphn:Patient` structure and missed the per-patient named graphs. *Fix:* an MCP server built on the SPHN VoID (182 classes, 980 aggregate patterns) plus a mapping from clinical words to classes.
- **Bigger was not better.** Qwen3-14B-AWQ never started on our vLLM setup. Qwen3-30B ran, but at a 4,096 context 3/6 requests failed with context-budget errors (moved to 8,192), and it brought no gain in correctness.
- **Reference answers were not a reliable oracle.** Some provided queries use properties missing from the 2026.1 schema (`hasRank` vs `hasRankCode`) or contradict their own prompt (example 3 filters `Principal` while the prompt asks for any rank). We flag these instead of silently "fixing" the labels.
- **A per-query threshold alone is not enough** (masked sums, differencing). This led to provenance tracking and the release history, which are only active in the hardened profile.
- **Throughput under evaluation.** Requests are serialised on one GPU, so the queue grew (p95 129 s) and most requests hit the 180 s timeout. Only 34/271 got a verdict.
- **The prompt guard never blocked.** Laya returned `allow` for every prompt; recall on malicious prompts was 0. The checker caught everything instead.
- **Known limits:** the checked runtime runs on local RDFLib and is not yet wired to Virtuoso; the release history lives in memory in one process; passing all checks does not certify anonymity or semantic equivalence.

# AI agents and LLM usage

We used LLM-based coding assistants and agentic coding tools heavily for scaffolding, refactoring, writing tests and documentation, and for drafting parts of this report. Every design decision (the two-gate architecture, the rule set, the attack corpus and its labels) was made and reviewed by the team, and all reported numbers come from our own runs. Inside the product, LLMs appear only as the Laya classifier and the SPARQL generator described above. The compliance verdict itself is deterministic.